

Formal Verification Report: powers_of_two_reach_one

Source: Self-authored demonstration problem, this session, no external publication

Black Swan Labs FCVE
ledger through EVENT-014

2026-09-19

Trust Statement. The formal result was checked using Lean 4.34.0 under the pinned project environment (Mathlib master 5ed2965). The theorem CLAIM-002 was kernel-checked with axiom footprint propext, Quot.sound. Independent checking status is INDEPENDENTLY_CHECKED (recorded before the Section 17 vocabulary existed, as PASS). Computational/adversarial testing produced: COMPUTATIONALLY_SUPPORTED; NO_COUNTEREXAMPLE_FOUND_IN_TESTED_DOMAIN (diagnostic only). The remaining trust limitations are: The repository, commit and toolchain were captured after the run (on 2026-09-19), not recorded during it; they describe the project as it is now; The checker used for the independent check is not identified in the record; only its result is; The Gate 8 search examined only $k = 0$ to 20 and $k = 50, 100, 500, 1000$. This report does not claim that computational testing constitutes formal proof or that formalization alone establishes the truth of the original informal statement.

Abstract

This report documents the formal verification of powers_of_two_reach_one (CLAIM-002) under the Formal Claim Verification Engine. Recorded: build PASS – 8924/8924 jobs, exit 0, no sorry/admit; axiom footprint propext, Quot.sound; independent check INDEPENDENTLY_CHECKED (recorded before the Section 17 vocabulary existed, as PASS). Computational testing is diagnostic and is not proof. Section 17 states the status the recorded evidence permits; the governance decision is recorded separately, after this report.

1 Mathematical Statement

powers_of_two_reach_one. *Every power of two reaches 1 under the standard Collatz function: for $f(n) = n/2$ if n even, $3n+1$ if n odd, and for every $k \geq 0$, iterating f starting from 2^k eventually reaches 1.*

Source: Self-authored demonstration problem, this session, no external publication. Source location: source/original.md.

2 Definitions and Assumptions

Definitions

- $f: \mathbb{N} \rightarrow \mathbb{N}$ (called collatzStep in the Lean formalization), $f(n) = n/2$ if n even, $3n+1$ if n odd
– this is the STANDARD (uncompressed) Collatz map, distinct from Eliahou’s compressed

map T used in VCE-001

Source assumptions

- **SA-001** k is a natural number, $k \geq 0$

Formalization assumptions

- **FA-001** 'iterating f m times' is represented via Mathlib's `Function.iterate (f^[m])`, a standard, well-understood construction with no hidden semantics beyond repeated function application

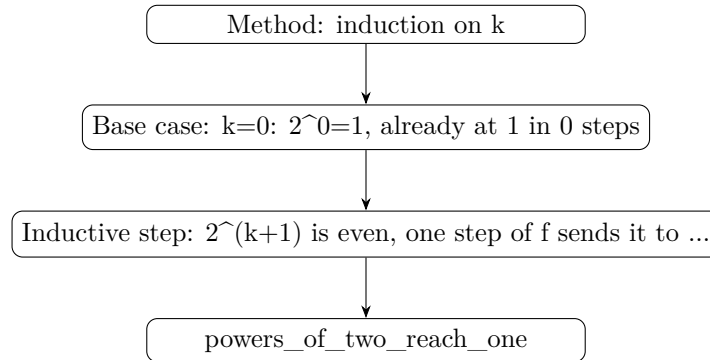
Computational assumptions

None recorded.

3 Proof Outline

1. **method.** induction on k
2. **base case.** $k=0$: $2^0=1$, already at 1 in 0 steps
3. **inductive step.** $2^{(k+1)}$ is even, one step of f sends it to 2^k ; by induction hypothesis 2^k reaches 1 in m more steps, so $2^{(k+1)}$ reaches 1 in $m+1$ steps

4 Proof Dependency Diagram



5 Formalization

Lean identifiers below are cross-references; the mathematics is in the sections above (§29).

`powers_of_two_reach_one`

`theorem powers_of_two_reach_one :  $\forall k : \mathbb{N}, \exists m : \mathbb{N}, (\text{collatzStep}^m) (2^k) = 1$`

6 Semantic Bridge

Informal Statement

Every power of two reaches 1 under the standard Collatz function: for $f(n) = n/2$ if n even, $3n+1$ if n odd, and for every $k \geq 0$, iterating f starting from 2^k eventually reaches 1.

Normalized Statement

Normalized Claim: CLAIM-002

Objects and domains

- $f : \mathbb{N} \rightarrow \mathbb{N}$, $f(n) = \begin{cases} n/2 & n \text{ even} \\ 3n+1 & n \text{ odd} \end{cases}$ — the **standard** (uncompressed) Collatz map. (Note: this is a different map from Eliahou’s compressed T used in VCE-001, which folds the forced halving after an odd step into one combined step. f and T are related but not identical.)
- $f^{(m)}$ denotes f composed with itself m times.

The claim

Given: $k \in \mathbb{N}$.

Conclusion: $\exists m \in \mathbb{N}$ such that $f^{(m)}(2^k) = 1$.

Why this is true

By induction on k .

- **Base case** ($k = 0$): $2^0 = 1$, so $m = 0$ works trivially.
- **Inductive step:** assume $f^{(m)}(2^k) = 1$ for some m . Since 2^{k+1} is even, $f(2^{k+1}) = 2^{k+1}/2 = 2^k$. So $f^{(m+1)}(2^{k+1}) = f^{(m)}(f(2^{k+1})) = f^{(m)}(2^k) = 1$.

This is a completely elementary, fully constructive argument — no case analysis beyond parity, no external results needed.

Status

MATCH — direct restatement, nothing to disambiguate.

Lean Statement

`theorem powers_of_two_reach_one :  $\forall k : \mathbb{N}, \exists m : \mathbb{N}, (\text{collatzStep}^{\wedge}[m]) (2^k) = 1$`

Differences

Gate 4: Lean Formalization Comparison — CLAIM-002

Human mathematics (normalized)	Lean
$f(n) = n/2$ if even, $3n + 1$ if odd	<code>def collatzStep (n : ℕ) : ℕ := if n % 2 = 0 then n / 2 else 3 * n + 1</code>
$\forall k, \exists m, f^{(m)}(2^k) = 1$	<code>theorem powers_of_two_reach_one : ∀ k : ℕ, ∃ m : ℕ, (collatzStep^[m]) (2 ^ k) = 1</code>
Induction on k , base case $2^0 = 1$, step: $2^{k+1} \rightarrow 2^k$ via one even step	<code>induction k with zero => exact ⟨0, rfl⟩ succ n ih => ... — identical structure</code>

Checklist

Check	Finding
Stronger/weaker	Neither — exact match
Altered domain	No
Missing/extra hypothesis	None
Changed quantifier	No
Changed function	No — <code>collatzStep</code> is exactly f as normalized
Coercion	None needed (pure $\mathbb{N}$ arithmetic throughout)
Finite/infinite mismatch	No

This is the simplest possible Gate 4 outcome: the Lean statement is a direct transliteration of the normalized claim, and the proof’s induction structure matches the normalized proof step-for-step (base case $k=0$, inductive step using `Function.iterate_succ` to peel off exactly one application of `collatzStep`, matching the normalized argument’s “ $f(2^{k+1}) = 2^k$, then apply the inductive hypothesis”).

Status: MATCH

Findings recorded by the reviewer

No difference was found by any of the required checks; each check records its basis in the semantic record.

Justification

Gate 4 records the Lean statement as a direct transliteration of the normalized claim (same function, domain, quantifiers, conclusion) and the proof’s induction structure matches the normalized argument step for step. The Gate 7 re-audit found no undisclosed change, and the axiom footprint contains no classical assumption.

Verdict

FAITHFUL

Set by Wilson. Recorded context: Gate 4 recorded MATCH; Gate 7 recorded PASS – no undisclosed semantic change; axiom footprint itself serves as independent corroboration of the re-audit’s conclusion.

7 Lean Environment

Lean Version	4.34.0
Mathlib Version	master 5ed2965

8 Formal Verification

PASS – 8924/8924 jobs, exit 0, no sorry/admit. Source of this field: ledger:EVENT-006.

9 Axiom Audit

Actual axiom footprint: propext, Quot.sound. Source: parsed-legacy-text:EVENT-007.

10 Independent Check

PASS – Checked 1662 declarations with no errors. Axioms confirmed: propext, Quot.sound (matches Gate 6 exactly). (legacy wording). Recording the checker and its version does not by itself establish independence (§17).

11 Computational / Adversarial Tests

Counterexample search: COMPUTATIONALLY_SUPPORTED – all cases reach 1 in exactly k steps as predicted; no counterexample in tested domain [diagnostic, not proof]

Computational test: NO_COUNTEREXAMPLE_FOUND_IN_TESTED_DOMAIN – 5001 cases over: every integer k in [0,5000]; full trajectory 2^k ; two independent implementations of f; NOT PROOF [diagnostic, not proof]

Neither result is formal proof (§16, §18).

What these tests do not establish

- They are not proof of the claim (Sections 16 and 18). A search that finds no counterexample is diagnostic evidence only.
- The tested domain is not recorded for these results, so this report cannot say which cases were examined.
- The Gate 8 search examined $k = 0$ to 20 and $k = 50, 100, 500$ and 1000 only; no other values of k were tested by that script. (stated by Wilson)

12 Verification Limitations

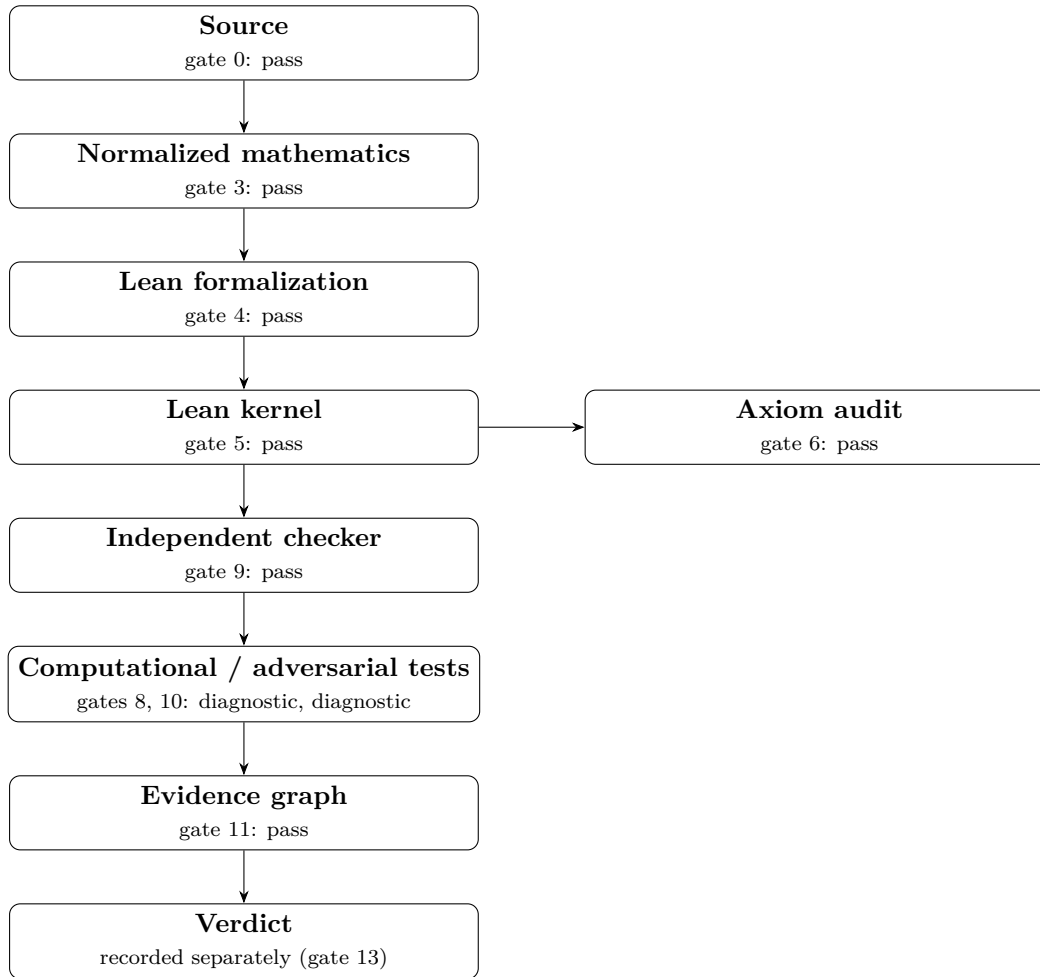
Item	Limitation
1	The repository, commit and toolchain were captured after the run (on 2026-09-19), not recorded during it; they describe the project as it is now.
2	The checker used for the independent check is not identified in the record; only its result is.

Item	Limitation
3	The Gate 8 search examined $k = 0$ to 20 and $k = 50, 100, 500$ and 1000 only; no other values of k were tested by that script (stated by Wilson).
<i>Record-keeping notes (these do not change what the verification shows)</i>	
4	10 of 14 ledger events predate the input/output hash fields and carry none.
5	Events EVENT-009 (COMPUTATIONALLY_SUPPORTED) used wording outside the Section 16 vocabulary; each is read as diagnostic evidence, not proof.

13 Verification Matrix

Gate	Ledger action	Event	Status	Satisfied
G0	source intake	EVENT-001	PASS	yes
G1	claim extraction	EVENT-002	PASS	yes
G2	assumption extraction	EVENT-003	PASS	yes
G3	semantic normalization	EVENT-004	PASS	yes
G4	lean formalization comparison	EVENT-005	PASS	yes
G5	lean build	EVENT-006	PASS	yes
G6	axiom audit	EVENT-007	PASS	yes
G7	semantic re audit	EVENT-008	PASS	yes
G8	adversarial computational test	EVENT-009	DIAGNOSTIC	yes
G9	independent check	EVENT-010	PASS	yes
G10	computational check	EVENT-011	DIAGNOSTIC	yes
G11	evidence graph	EVENT-012	PASS	yes
G12	report generation	EVENT-014	PASS	yes
ORDER	canonical chain (§2)	-	PASS	yes

14 Trust / Verification Chain



15 Reproducibility Instructions

Repository URL	NOT RECORDED
Commit	409c4c3b6f0179ae9131ac2450998e59a7fb1132 (working tree clean)
Project directory (now)	/Users/lordwilson/ico-collatz/ico_collatz_verification
Lean toolchain file	leanprover/lean4:v4.34.0
Lean version	4.34.0
Lake version	Lake version 5.0.0-src+293d5d0 (Lean version 4.34.0)
Mathlib revision	master 5ed2965 (commit 5ed2965256430c3649e86755f9576b54eca72435)
Checker	NOT RECORDED
Expected build result	PASS – 8924/8924 jobs, exit 0, no sorry/admit
How these were obtained	Captured 2026-09-19 from the project as it is now; not recorded during the run
Matches the commit the run's notes name	not checked

the commit is local only (no remote recorded): reproduce from a copy of the project repository,

```

then: git checkout 409c4c3b6f0179ae9131ac2450998e59a7fb1132
cat lean-toolchain
lake --version
lean --version
lake build IcoCollatzVerification.PowersOfTwoReachOne
# in a Lean file importing the module: #print axioms powers_of_two_reach_one
# independent-check commands were not recorded for this run

```

The repository facts were captured after the run and describe the project as it is now; they do not prove what the original run built. The build command is the one recorded in EVENT-006. Reproducibility level supported by recorded evidence: R4. This tool did not re-run the build.

16 Correction Ledger

No failed or disputed gate results are recorded in the ledger.

Correction Ledger — VCE-002 rev s

CORRECTION-001 — report EVENT-014 superseded: local-only commit described as “not recorded”

- **original_claim:** the Reproducibility Instructions of the first rev-s report were accurate.
- **error:** with a clean project at a commit but no remote, the report listed the commit in its table and then printed the comment “repository location and commit were not recorded for this run (see table above)”, which contradicts the table.
- **detection_method:** reading the rendered PDF.
- **root_cause:** the reproduction-instructions builder had no branch for CLEAN_AT_COMMIT without a remote and fell through to the “nothing recorded” default.
- **repair:** a branch now says the commit is local only and gives the `git checkout target` without a clone command; regression test added (`test_clean_local_only_commit_is_not_called_unrecorded_and_gets_no_clone_command`). The report was regenerated; EVENT-014 stays in the ledger, superseded; its files are kept in `report-superseded-EVENT-014/`.
- **recheck:** `tests/test_fcve_report.py` passes; the regenerated PDF was read again.
- **result:** RESOLVED. Nothing was deleted. The facts (commit 409c4c3b6f01..., clean, no remote) were correct throughout; only the comment was wrong.
- **date:** 2026-09-19

Note — what rev s changes relative to rev r (not a correction)

Rev r said the repository, commit and toolchain “were never recorded”. Rev s cites a reproduction snapshot of the project at commit 409c4c3b..., taken after the runs (the project was committed on 2026-09-19). It identifies the source as it is now; it does not prove what the original 2026-09 runs built. The checker is still not identified in the record.

17 Conclusion

The recorded evidence is summarized in the Trust Statement and the verification matrix above. Recorded failure is not a passed gate.

FORMAL PROOF (build):	PASS
AXIOM AUDIT:	PASS
SEMANTIC MATCH (Gate 4):	PASS
SEMANTIC RE-AUDIT (Gate 7):	PASS
INDEPENDENT CHECK:	PASS
COUNTEREXAMPLE SEARCH:	NO COUNTEREXAMPLE FOUND IN TESTED DOMAIN (diagnostic, not proof)
COMPUTATIONAL TEST:	NO COUNTEREXAMPLE FOUND IN TESTED DOMAIN (diagnostic, not proof)
REPRODUCIBILITY:	R4 (ceiling supported by recorded evidence)

Status the evidence permits. The recorded evidence satisfies the conditions for promotion (Section 53). Whether to promote is the reviewer’s decision, recorded after this report. This is not a decision: the governance decision is recorded as a separate step after this report.

A Lean Source

```
theorem powers_of_two_reach_one :  $\forall k : \mathbb{N}, \exists m : \mathbb{N}, (\text{collatzStep}^m) (2^k) = 1$ 
```

B Tests

- **EVENT-009** python3 verification-002/computational/tests/verify_powers_of_two.py: COMPUTATIONALLY_SUPPORTED – all cases reach 1 in exactly k steps as predicted; no counterexample in tested domain
- **EVENT-011** python Python 3.12.9: verify_gate10.py: NO_COUNTEREXAMPLE_FOUND_IN_TESTED_DOMAIN – 5001 cases over: every integer k in $[0, 5000]$; full trajectory 2^{k+1} ; two independent implementations of f ; NOT PROOF

C Evidence Receipt

Field	Value
SOURCE HASH	sha256:97d49fbc3576e79f3901ce85f0fb9dd92d9cbf06d6be933aada6a2f1a194d60 (../verification-002/source/original.md)
THEOREM ID	CLAIM-002 (powers_of_two_reach_one)
CLAIM	Every power of two reaches 1 under the standard Collatz function: for $f(n) = n/2$ if n even, $3n+1$ if n odd, and for every $k \geq 0$, iterating f starting from 2^k eventually reaches 1.

Field	Value
NORMALIZED CLAIM	verification-002/normalized/normalized-claim.md – MATCH
LEAN STATEMENT	Gate 4: MATCH
LEAN VERSION	4.34.0
MATHLIB VERSION	master 5ed2965
BUILD RESULT	PASS – 8924/8924 jobs, exit 0, no sorry/admit
AXIOM FOOTPRINT	propext, Quot.sound
INDEPENDENT CHECK	PASS – Checked 1662 declarations with no errors. Axioms confirmed: propext, Quot.sound (matches Gate 6 exactly). (legacy wording)
COMPUTATIONAL TEST	NO_COUNTEREXAMPLE_FOUND_IN_TESTED_DOMAIN – 5001 cases over: every integer k in $[0, 5000]$; full trajectory 2^k ; two independent implementations of f ; NOT PROOF [diagnostic, not proof]
COUNTEREXAMPLE SEARCH	COMputationALLY_SUPPORTED – all cases reach 1 in exactly k steps as predicted; no counterexample in tested domain [diagnostic, not proof]
SEMANTIC MATCH	Gate 4: MATCH; Gate 7: PASS – no undisclosed semantic change; axiom footprint itself serves as independent corroboration of the re-audit’s conclusion
REPRODUCIBILITY LEVEL	R4 (ceiling supported by recorded evidence; NOT re-run by this tool; R5 needs full-package rebuild, not assessed)
GRAPH HASH	v3 f77904a7ca276a85fbf25bbdf43df7f7ed501d91741b4e5e0b22f8c0e1467b58 (covers 14 events through EVENT-014)
COST	14 ledger events; LEAN_BUILD_TIME=NOT RECORDED; COMPUTATIONAL_TEST_TIME=NOT RECORDED; CHECKER_TIME=NOT RECORDED; HUMAN_TIME, MODEL_CALL_COUNT, MODEL_COST, LATEX_BUILD_TIME=NOT RECORDED (no cost ledger, §43)